

LAPORAN CODELAB

OKTOBER 2025



PEMROGRAMAN FUNGSIONAL

MODUL 3 - Processing Sequence Data in Functional Programming

DISUSUN OLEH:

Muhammad Yusron Al Ghoni Rizqullah

202310370311333

Fungsional D

1. List Comprehension

Latihan 1 : jawaban dari Latihan 1 - Logika dasar

Screenshot jawaban (harus mengandung nama-nim):

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

list_ganjil = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47,
print(list_ganjil)
```

```
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49]
```

```
# Muhammad Yusron AL Ghoni Rizqullah
# 202310370311333
```

```
list_ganjil = []
list_genap = []

#imperatif loop
for i in range(50):
    if i % 2:
        list_ganjil.append(i)
    else:
        list_genap.append(i)
```

```
print(list_ganjil)
print(list_genap)
```

```
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49]
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
```

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333
```

```
list_ganjil = [x for x in range (50) if x % 2]
list_genap = [x for x in range (50) if not x % 2]
print(list_genap)
print(list_ganjil)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43, 45, 47, 49]
```

Penjelasan :

Jadi dalam Python, Kode diatas yang seperti “[x for x in range(50) if x % 2]” dan [x for x in range(50) if not x % 2]” dapat berfungsi tanpa operator perbandingan dikarenakan Python memperlakukan nilai non-zero (bukan nol) sebagai True dan nilai nol (0) sebagai False dalam konteks boolean. Untuk list list_ganjil, kondisi if x % 2 akan bernilai True ketika x % 2 menghasilkan 1 (untuk angka ganjil), dan bernilai False ketika x % 2 menghasilkan 0 (untuk angka genap). Sebaliknya, untuk list_genap, kondisi if not x % 2 akan bernilai True ketika x % 2 menghasilkan 0 (yaitu not False), dan bernilai False ketika x % 2 menghasilkan 1 (yaitu not True). Memanfaatkan sifat *truthiness* ini membuat kode menjadi lebih ringkas dan ekspresif, yang merupakan penulisan atau gaya yang umum digunakan dalam pemrograman Python.

2. Nested List Percobaan 1

Latihan 2 : Gunakan list comprehension untuk membuat matrix 3x4 dengan isi angka berurutan dari 0 hingga 11. Dan ditampilkan dalam bentuk matrix:

```
[0, 1, 2, 3]
[4, 5, 6, 7]
[8, 9, 10, 11]
```

Screenshot jawaban (harus mengandung nama-nim) :

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333
matrix1 = []

for i in range(4): # 4 baris
    matrix1.append([])
    for j in range(5): # 5 kolom
        matrix1[i].append(j)

print(matrix1)

[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

✖ Latihan 2 - Percobaan 1

Gunakan list comprehension untuk membuat matrix 3x4 dengan isi angka berurutan dari 0 seperti berikut!

```
[0, 1, 2, 3]
[4, 5, 6, 7]
[8, 9, 10, 11]
```

kalian bisa memodifikasi list comprehension pada matrix2 untuk ini

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333
matrix_latihan2 = [[j + i * 4 for j in range(4)] for i in range(3)]

for row in matrix_latihan2:
    print(row)

[0, 1, 2, 3]
[4, 5, 6, 7]
[8, 9, 10, 11]
```

Penjelasan :

Kode pada Latihan 2 menggunakan nested list comprehension untuk secara efisien membuat matriks berukuran 3 baris (dari `i in range(3)`) dan 4 kolom (dari `j in range(4)`). Bagian dalam `[j + i * 4 for j in range(4)]` menghasilkan setiap baris matriks. Di sini, `j` mewakili indeks kolom (0-3), dan `i` mewakili indeks baris (0-2). Dengan menambahkan `i * 4`, setiap elemen dihitung berdasarkan posisi baris dan kolomnya,

memastikan angka berurutan dari 0 hingga 11 ($0 + 0*4 = 0$, $1 + 0*4 = 1$, ..., $0 + 1*4 = 4$, ..., $3 + 2*4 = 11$). List comprehension luar kemudian mengulangi proses ini untuk setiap baris, menghasilkan matriks 3x4 yang diinginkan.

3. Nested List Percobaan 2

Latihan 3a : jawaban dari pertanyaan latihan 3a - Percobaan 2

Screenshot jawaban (harus mengandung nama-nim) :

```
# Muhammad Yusron AL Ghoni Rizqullah
# 202310370311333
```

```
matrix = []
for y in range(baris):
    row = []
    for x in range(kolom):
        row.append(x+y*kolom)
    matrix.append(row)
```

```
print("Matriks asli:")
for row in matrix:
    print(row)
```

```
⇒ Matriks asli:
[0, 1, 2, 3, 4]
[5, 6, 7, 8, 9]
[10, 11, 12, 13, 14]
```

Dengan list Comperhension :

```
# Muhammad Yusron AL Ghoni Rizqullah
# 202310370311333
```

```
# membuat matrix dg list comprehension
matrix = [[x+y*kolom for x in range(kolom)] for y in range(baris)]
```

```
print("Matriks asli:")
for row in matrix:
    print(row)
```

```
⇒ Matriks asli:
[0, 1, 2, 3, 4]
[5, 6, 7, 8, 9]
[10, 11, 12, 13, 14]
```

Penjelasan :

Kedua set kode pada Percobaan 2, meskipun berbeda sintaks, menghasilkan output yang identik karena menerapkan logika pembuatan dan pemfilteran matriks yang setara. Kode tanpa list comprehension secara eksplisit menggunakan nested for loop dan metode append() untuk membangun matriks baris demi baris, sementara list

comprehension (`[[j for j in range(5)] for i in range(4)]`) mencapai hasil yang sama dengan sintaks yang lebih ringkas dan deklaratif. List comprehension mengabstraksi detail iterasi manual dan penambahan elemen, memungkinkan kita untuk menyatakan *apa* yang ingin kita buat (matriks 4x5 berisi angka 0-4 di setiap baris) alih-alih *bagaimana* cara membuatnya langkah demi langkah. Proses filtering dengan `if` juga mengikuti prinsip yang sama; kondisi filter diterapkan pada setiap elemen (inner loop) atau setiap baris (outer loop) secara logis setara di kedua pendekatan, sehingga menghasilkan matriks yang difilter dengan struktur dan isi yang sama.

Latihan 3b : Hasil modifikasi kode Percobaan 2, dan jawaban dari perbedaan filtering pada inner loop dan outer loop dengan contoh sederhana

Screenshot jawaban (harus mengandung nama-nim) :

Sesudah dimodifikasi :

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

# Latihan 3b - Percobaan 2 Modified
matrix_latihan3b = [[j + i * 6 + 1 for j in range(6)] for i in range(4)]

print("Matriks asli:")
for row in matrix_latihan3b:
    print(row)

hasil_filtering = [[j + i * 6 + 1 for j in range(6) if (j + i * 6 + 1) % 2 != 0] for i in range(4) if i % 2 == 0]

print("\nHasil filtering (bilangan ganjil dari baris genap):")
for row in hasil_filtering:
    print(row)

Matriks asli:
[1, 2, 3, 4, 5, 6]
[7, 8, 9, 10, 11, 12]
[13, 14, 15, 16, 17, 18]
[19, 20, 21, 22, 23, 24]

Hasil filtering (bilangan ganjil dari baris genap):
[1, 3, 5]
[13, 15, 17]
```

Penjelasan :

Pada dasarnya, filtering pada inner loop akan memengaruhi elemen-elemen di dalam setiap sublist (kolom), sedangkan filtering pada outer loop akan memengaruhi sublist itu sendiri (baris).

Jadi berikut ini untuk detailnya :

- **Filtering pada Inner Loop:** Kondisi `if` diletakkan setelah `for` yang kedua (inner loop). Ini berarti kondisi tersebut akan dievaluasi untuk setiap elemen individual dalam sublist. Jika kondisi bernilai `True`, elemen tersebut akan dimasukkan ke dalam sublist yang baru. Hasilnya adalah nested list dengan sublist yang mungkin memiliki panjang yang bervariasi, tergantung pada elemen mana yang lolos filter di setiap baris.
- **Filtering pada Outer Loop:** Kondisi `if` diletakkan setelah `for` yang pertama (outer loop). Ini berarti kondisi tersebut akan dievaluasi untuk setiap sublist (baris) secara keseluruhan. Jika kondisi bernilai `True`, seluruh sublist tersebut akan dimasukkan ke dalam list baru. Jika kondisi bernilai `False`, seluruh sublist (baris) tersebut akan diabaikan. Hasilnya adalah nested list dengan jumlah sublist (baris) yang lebih sedikit dari aslinya, tetapi panjang setiap sublist

tetap sama seperti aslinya (kecuali jika ada filtering tambahan di inner loop).

4. Filter

Latihan 4 : Membuat filter untuk menyaring produk yang stoknya habis dan menampilkan dengan iterasi

Screenshot jawaban (harus mengandung nama-nim) :

```
#percobaan latihan 4

#percobaan latihan 4
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

def stok_habis(produk_item):
    return produk_item["stok"] == 0

produk_habis = filter(stok_habis, produk)

print("Produk dengan stok habis:")
for item in produk_habis:
    print(item)
```

➡ Produk dengan stok habis:
{'nama': 'Mouse', 'stok': 0}
{'nama': 'Monitor', 'stok': 0}

Penjelasan :

Kode di atas menggunakan fungsi `filter()` untuk menyaring data `produk`.

1. Pertama, kita mendefinisikan fungsi `stok_habis(produk_item)` yang bertindak sebagai fungsi logika. Fungsi ini akan mengembalikan `True` jika nilai `"stok"` dari `produk_item` adalah 0, dan `False` jika sebaliknya.
2. Kemudian, fungsi `filter()` dipanggil dengan argumen `stok_habis` (fungsi logika) dan `produk` (iterable yang akan disaring). Hasilnya adalah objek filter (`produk_habis`) yang berisi elemen-elemen dari `produk` di mana fungsi `stok_habis` mengembalikan `True`.
3. Terakhir, kita melakukan iterasi pada objek `produk_habis` untuk menampilkan setiap produk yang stoknya habis.

Penggunaan `filter()` di sini menunjukkan bagaimana cara menyaring data secara deklaratif dan fungsional, di mana kita hanya menyatakan kriteria penyaringan (`stok_habis`) tanpa perlu menuliskan perulangan `for` secara eksplisit untuk memeriksa setiap elemen.

5. Map

Latihan 5 : hasil dan pengamatan latihan 5 - Logika map

Screenshot jawaban (harus mengandung nama-nim):

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

grades = map(to_grade, nilai)

print("Nilai dengan Grade:")
# iterasi untuk menampilkan data grades menggunakan next()
for score in nilai:
    print(score, '=', next(grades))

#percobaan latihan 5
```

↩ Nilai dengan Grade:

```
55 = C
80 = B
72 = B
65 = C
90 = A
45 = C
88 = A
60 = C
90 = A
50 = C
73 = B
```

Penjelasan :

Pada Latihan 5, loop `for score in nilai:` digunakan untuk menampilkan setiap nilai dari list nilai beserta gradenya yang dihasilkan oleh `map(to_grade, nilai)`. Meskipun iterasi dilakukan pada list nilai, fungsi `next(grades)` dipanggil di setiap iterasi untuk mengambil elemen berikutnya dari objek map yang bersifat *lazy*. Ini aman karena jumlah elemen dalam objek map (`grades`) akan selalu sama dengan jumlah elemen dalam nilai, sehingga tidak akan ada data yang terlewat atau terjadi `StopIteration exception` selama iterasi masih dalam batas jumlah elemen pada nilai.

6. Reduce

Latihan 6a : jalankan program yang ada, tampilkan dan amati hasilnya!

Screenshot jawaban (harus mengandung nama-nim):

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333
|
#percobaan latihan 6a
reduce1 = reduce(add, nilai_mahasiswa)
print(reduce1)

[80, 75, 90, 60, 70, 65, 85, 88, 92]
```

Penjelasan :

Kode ini menghasilkan error karena fungsi `reduce()` dengan fungsi `add()` (yang pada dasarnya adalah operator `+`) tidak dapat langsung diterapkan pada `nilai_mahasiswa` yang merupakan nested list. Fungsi `add()` hanya bisa menggabungkan dua elemen, dan pada iterasi pertama, `reduce` mencoba menggabungkan elemen pertama dari `nilai_mahasiswa` (yaitu list `[80, 75, 90]`) dengan elemen kedua (list `[60, 70, 65]`), yang mana operasi penjumlahan (`+`) antara dua list tidak menghasilkan nilai tunggal seperti yang diharapkan oleh `reduce` untuk mereduksi koleksi data menjadi satu nilai.

Latihan 6b : kode penyelesaian dan penjelasan program

Screenshot jawaban (harus mengandung nama-nim):

```
[74]
✓ od #percobaan latihan 6b
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

# Pertama, ratakan nested list menjadi satu list tunggal
flattened_nilai = reduce(add, nilai_mahasiswa)

# Kemudian, gunakan reduce untuk menjumlahkan semua angka dalam list tunggal
total_nilai_keseluruhan = reduce(add, flattened_nilai)

print("Total nilai keseluruhan:", total_nilai_keseluruhan)

Total nilai keseluruhan: 705
```

Penjelasan :

Untuk menjumlahkan semua angka dalam nested list `nilai_mahasiswa` menjadi satu nilai tunggal, kita perlu meratakannya terlebih dahulu menjadi satu list tunggal. Ini dilakukan dengan menggunakan `reduce(add, nilai_mahasiswa)`. Karena operator `+` pada list berfungsi sebagai penyambung (concatenation), `reduce` akan menggabungkan semua sublist menjadi satu list besar. Setelah nested list diratakan, kita dapat menggunakan `reduce(add, flattened_nilai)` sekali lagi untuk menjumlahkan semua elemen angka dalam list tunggal tersebut, menghasilkan total nilai keseluruhan.

Codelab 1

7. List Comprehension

1.:

Gunakan list comprehension untuk mengambil huruf vokal atau konsonan dari namaLengkap. (todo 1)

Screenshot jawaban (harus mengandung nama-nim)

```
[84]
✓ 0 d      # Muhammad Yusron Al Ghoni Rizqullah
           # 202310370311333
           namaLengkap = "muhammad yusron al ghoni rizqullah" # huruf kecil semua

1. Gunakan list comprehension untuk mengambil huruf vokal atau konsonan dari namaLengkap.

[85]
# TODO 1: Ambil huruf vokal (untuk NIM genap) atau konsonan (untuk NIM ganjil)
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

vokal = [char for char in namaLengkap if char in 'aeiou']
konsonan = [char for char in namaLengkap if char not in 'aeiou' and char != ' ']

[86]
✓ 0 d      print("Huruf vokal:", vokal)
           print("Huruf konsonan:", konsonan)

Huruf vokal: ['u', 'a', 'a', 'u', 'o', 'a', 'o', 'i', 'i', 'u', 'a']
Huruf konsonan: ['m', 'h', 'm', 'm', 'd', 'y', 's', 'r', 'n', 'l', 'g', 'h', 'n', 'r', 'z', 'q', 'l', 'l', 'h']
```

Penjelasan :

Pada langkah ini, list comprehension digunakan untuk secara efisien membuat dua list baru, yaitu vokal dan konsonan, dari string namaLengkap. Sintaks [char for char in namaLengkap if ...] memungkinkan kita untuk mengiterasi setiap karakter dalam string dan menyertakannya ke dalam list baru hanya jika karakter tersebut memenuhi kondisi yang diberikan dalam klausa if. Kondisi ini memeriksa apakah karakter merupakan huruf vokal atau konsonan (dengan mengabaikan spasi) sesuai dengan NIM (genap untuk vokal, ganjil untuk konsonan).

8. Map

2.:

Lengkapi fungsi transformasi double(x) berikut dan gunakan untuk memetakan variabel vokal/konsonan kalian menjadi data baru dan tampilkan dalam bentuk list untuk data vokal dan tuple untuk data konsonan! (todo 2 & 3)

Screenshot jawaban (harus mengandung nama-nim)

```
# TODO 2: Melengkapi fungsi sesuai deskripsi
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

def double(x):
    return x * 2

# TODO 3: Gunakan fungsi double untuk memetakan data vokal/konsonan
new_data = list(map(double, konsonan))

print("Data konsonan setelah didouble (sebagai tuple):", tuple(new_data))

Data konsonan setelah didouble (sebagai tuple): ('mm', 'hh', 'mm', 'mm', 'dd', 'yy', 'ss', 'rr', 'nn', 'll', 'gg', 'hh', 'nn', 'rr', 'zz', '')
```

Penjelasan :

TODO 2 meminta kita untuk melengkapi fungsi double(x) yang akan mengembalikan nilai input dikalikan dua. Fungsi ini merupakan fungsi transformasi murni karena hanya bergantung pada inputnya dan tidak memiliki efek samping. TODO 3 kemudian menggunakan fungsi map() bersama dengan fungsi double() untuk menerapkan transformasi penggandaan pada setiap elemen dalam list konsonan (sesuai dengan NIM ganjil). Hasil dari map() adalah objek map yang *lazy*, sehingga perlu diubah menjadi list atau tuple (sesuai ketentuan soal) untuk ditampilkan.

9. Filter

3.:

Siapkan fungsi logika untuk digunakan dalam fungsi filter() bersamaan dengan data hasil map dengan tujuan untuk mengambil huruf tertentu (bebas sesuai NIM). (todo 4-5)

Screenshot jawaban (harus mengandung nama-nim)

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

# TODO 4: Definisikan fungsi logika untuk memilih huruf tertentu
def is_a(char):
    return char == 'a'

# TODO 5: Penggunaan fungsi filter untuk memfilter data hasil map sebelumnya
# Menggunakan data konsonan hasil double
hasil_filter = filter(is_a, new_data)

#contoh hasil filter vokal aa
print("Hasil filter huruf 'a' dari data konsonan setelah didouble (diubah ke list):", list(hasil_filter))

Hasil filter huruf 'a' dari data konsonan setelah didouble (diubah ke list): []

#contoh hasil filter konsonan nn
# Tidak relevan dengan filtering huruf 'a' dari konsonan double. Dibiarkan kosong atau dihapus jika tidak diperlukan.
```

Penjelasan :

TODO 4 mendefinisikan fungsi logika is_a(char) yang bertugas memeriksa apakah sebuah karakter adalah huruf 'a'. Fungsi ini mengembalikan True atau False, menjadikannya cocok untuk filtering. TODO 5 kemudian menggunakan fungsi filter() dengan fungsi logika is_a dan data new_data (hasil dari map) untuk menyaring elemen-elemen yang memenuhi kriteria (yaitu, yang merupakan huruf 'a'). Hasilnya adalah objek filter yang *lazy*, yang kemudian diubah menjadi list untuk ditampilkan.

10. Reduce

4.:

Gunakan fungsi `reduce()` dari modul `functools` untuk menghitung berapa kali huruf tertentu ('a' atau huruf lain sesuai NIM) muncul dalam daftar huruf vokal/konsonan.

! dilarang menggunakan data hasil filter. (todo 6-8)

Screenshot jawaban (harus mengandung nama-nim)

```
# Muhammad Yusron Al Ghoni Rizqullah
# 20231037011333
# TODO 6: Import reduce
from functools import reduce

# TODO 7: Definisikan fungsi reduce untuk menghitung jumlah huruf tertentu
def count_char(count, char):
    if char == 'a':
        return count + 1
    return count

# TODO 8: Gunakan reduce() untuk menghitung total kemunculan huruf
jumlahA = reduce(count_char, konsonan, 0) # vokal atau konsonan

# contoh output
print("Jumlah huruf 'a' dalam konsonan:", jumlahA)

Jumlah huruf 'a' dalam konsonan: 0
```

Penjelasan :

TODO 6 mengimpor fungsi `reduce` dari modul `functools` yang diperlukan untuk mereduksi koleksi data menjadi satu nilai tunggal. TODO 7 mendefinisikan fungsi `count_char(count, char)` yang berfungsi sebagai fungsi reduksi. Fungsi ini mengambil dua argumen: akumulator `count` dan elemen saat ini `char`. Logikanya adalah menambahkan 1 ke `count` jika `char` adalah huruf 'a', dan mengembalikan `count` saat ini jika bukan. TODO 8 menggunakan `reduce()` dengan fungsi `count_char` dan list `konsonan` (data asli sebelum di-'double' atau di-filter, sesuai instruksi latihan), dengan nilai awal akumulator 0, untuk menghitung total kemunculan huruf 'a' dalam list `konsonan`.

Codelab 2

11. Recursive

Codelab 2 : jawaban codelab 2 dan jelaskan base case dan recursive case nya bagaimana

Screenshot jawaban (harus mengandung nama-nim)

```
# Muhammad Yusron Al Ghoni Rizqullah
# 202310370311333

# TODO 1: Buat fungsi rekursif cek_palindrom(teks)
def cek_palindrom(teks):
    if len(teks) <= 1:
        return True
    elif teks[0] == teks[-1]:
        return cek_palindrom(teks[1:-1])
    else:
        return False

print(f"'katak' adalah palindrom: {cek_palindrom('katak')}")
print(f"'aku' adalah palindrom: {cek_palindrom('aku')}")
print(f"'ada' adalah palindrom: {cek_palindrom('ada')}")
```

'katak' adalah palindrom: True
'aku' adalah palindrom: False
'ada' adalah palindrom: True

Penjelasan :

Fungsi `cek_palindrom(teks)` menggunakan rekursi untuk menentukan apakah sebuah string adalah palindrom. *Base case* tercapai ketika panjang string kurang dari atau sama dengan 1, karena string kosong atau string dengan satu karakter selalu dianggap palindrom. *Recursive case* memeriksa apakah karakter pertama (`teks[0]`) dan karakter terakhir (`teks[-1]`) dari string sama. Jika sama, fungsi memanggil dirinya sendiri (`cek_palindrom(teks[1:-1])`) dengan substring yang tidak menyertakan karakter pertama dan terakhir, secara efektif "memotong" string dari kedua ujungnya. Proses ini berlanjut hingga mencapai *base case* atau menemukan sepasang karakter pertama dan terakhir yang berbeda, yang mengindikasikan bahwa string tersebut bukan palindrom.

RUBRIK PENILAIAN CODELAB

Soal Latihan	Poin maksimal
1	5
2	10
3	10
4	10
5	10
6	10
7	5
8	10
9	5
10	10
11	15
Total	100

PENGURANGAN NILAI:

1. Jika **Screenshot jawaban** tidak mengandung identitas praktikan ybs (-2) per jawaban
2. Jika **Penjelasan** jawaban ditemukan identik dengan praktikan lain (-5) per soal
3. Berlaku untuk masing-masing soal yang dirasa tidak memenuhi poin diatas